

The Cipher Breach

Category : Cryptography

Difficulty : Medium

Author : Radhika Suryawanshi

This writeup details the solution to a three-part cryptographic challenge. Each stage exploits a common real-world vulnerability to recover a piece of the final flag.

Stage 1: ECDSA Nonce Leak

- **Vulnerability:** The challenge provides a list of ECDSA signatures where the top 12 bits of the secret nonce (k) are leaked. The security of ECDSA relies on this nonce being completely secret. Leaking even a few bits is catastrophic.
- **Attack:** The attack is to brute-force the remaining unknown bits of the nonce. By taking any two signatures, we can guess the full nonce for the first signature, calculate a potential private key, and then verify if that key is correct by checking it against the second signature's leaked nonce bits.

▪ Solver: solve_stage1.py

```
(radhika23s@kali)-[~]
└─$ # Update your package list and upgrade packages
sudo apt update && sudo apt upgrade -y

# Install Python, pip, and necessary build tools
sudo apt install -y python3 python3-pip python3-venv build-essential libgmp-dev

# Create and activate a Python virtual environment
python3 -m venv ~/ctf-env
source ~/ctf-env/bin/activate

# Install the required Python libraries for the challenges
pip install pycryptodome ecdsa pillow

# Create a main directory for your CTF
mkdir -p ~/questCON_ctf && cd ~/questCON_ctf
[sudo] password for radhika23s:
Get:1 https://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 https://kali.download/kali kali-rolling/main amd64 Packages [20.9 MB]
Get:3 https://kali.download/kali kali-rolling/main amd64 Contents (deb) [51.6 MB]
Fetched 72.6 MB in 10s (7,184 kB/s)
10 packages can be upgraded. Run 'apt list --upgradable' to see them.
The following packages were automatically installed and are no longer required:
  amass-common          libgeos3.13.1      libogdi4.1         libsigsegv2        libx264-164          python3-gpg          python3-kismetcaptureertlamr
  firmware-ti-connectivity libjs-jquery-ui    libplacebo349     libtheora0         libyelp0             python3-kismetcaptureertbtgeiger python3-kismetcaptureertlamr
  libbluray2           libjs-underscore  libportmidi0     libtheoradec1     linux-image-6.12.25-amd64 python3-kismetcapturefreaklabszigbee samba-ad-dc
  libbson-1.0-0t64     libmongoc-1.0-0t64 libravie0.7      libtheoraenc1     python3-bluepy       python3-kismetcaptureertl433      samba-ad-provision
  libgdal36            libmongocrypt0    libsframe1       libudfread0       python3-click-plugins python3-kismetcaptureertladsb      samba-dsdb-modules
Use 'sudo apt autoremove' to remove them.

Upgrading:
  iproute2  libpocl2-common  libpocl2t64  libyaml-cpp0.8  pocl-opencl-icd  pyinstaller  python3-lxml  python3-pyinstaller  python3-ujson
```

```

(ctf-env)-(radhika23s@kali)-[~/questCON_ctf]
$ # This command creates the correctly formatted solver script.
cat > stage1_solver.py <<'py'
#!/usr/bin/env python3
import json, hashlib

# The order of the secp256k1 curve
n = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141

def solve_ecdsa_nonce_leak(signatures, leaked_bits):
    t = leaked_bits
    s1_data = signatures[0]
    s2_data = signatures[1]

    r1, s1 = int(s1_data['r'], 16), int(s1_data['s'], 16)
    h1 = int.from_bytes(hashlib.sha256(bytes.fromhex(s1_data['msg']))).digest(), 'big')
    l1 = int(s1_data['leaked_top_bits'], 16)

    r2, s2 = int(s2_data['r'], 16), int(s2_data['s'], 16)
    h2 = int.from_bytes(hashlib.sha256(bytes.fromhex(s2_data['msg']))).digest(), 'big')
    l2 = int(s2_data['leaked_top_bits'], 16)

    # Brute-force the remaining unknown bits of the first nonce, k1
    for i in range(2**t):
        k1_guess = (l1 << (256 - t)) + i

        try:
            # Calculate a guess for the private key 'd'
            r1_inv = pow(r1, -1, n)
            d_guess = ((s1 * k1_guess - h1) * r1_inv) % n
        except ValueError:
            continue

        # Verify our d_guess using the second signature
        try:
            # Calculate what k2 should be if d_guess is correct
            k2_calculated = (pow(s2, -1, n) * (h2 + r2 * d_guess)) % n
        except ValueError:
            continue

        # Check if the top bits of our calculated k2 match the leaked ones
        if (k2_calculated >> (256 - t)) == l2:
            print(f"✓ Private key found!")
            return d_guess

```

```

    return None

— Main Execution —
# Open the file and access the "signatures" list inside the JSON
with open("mindflayer_logs.json", 'r') as f:
    signatures_data = json.load(f)

priv_key = solve_ecdsa_nonce_leak(signatures_data, 12)

if priv_key:
    part1 = hashlib.sha256(b"mindflayer_key:" + str(priv_key).encode()).hexdigest()[:12]
    print(f"Recovered Private Key: {hex(priv_key)}")
    print(f"Flag Part 1: {part1}")
else:
    print("✗ Failed to recover private key.")
PY

```

```

(ctf-env)-(radhika23s@kali)-[~/questCON_ctf]
$ python3 stage1_generator.py
python3 stage1_solver.py
✓ Stage 1 generated. Flag part: f39cd9720c4b
✓ Private key found!
Recovered Private Key: 0xca21ebfa051bda43c9c986b434e8020430f736e6d649c2e1e590714ff0453077
Flag Part 1: 5c8fd51d18da

```

Stage 2: MT19937 State Cloning

- **Vulnerability:** The Mersenne Twister (MT19937) is a non-cryptographically secure PRNG. Its internal state can be completely recovered from just 624 consecutive outputs. The outputs in `mt_observations.bin` are **tempered** (scrambled), so they must be **untempered** first.
- **Attack:** We read the 624 outputs, apply a known "untempering" algorithm to each one to find the raw internal state, and then seed our own MT19937 implementation with this state. We can then predict the next four numbers, which form the 16-byte AES key needed to decrypt `encrypted_blob.bin`.
- **Solver:** `solve_stage2.py`

```
(ctf-env)-(radhika23s@kali)-[~/questCON_ctf]
$ cat > stage2_generator.py <<'PY'
#!/usr/bin/env python3
# stage2_generator.py: Eleven's Randomness

import random
import struct
import os
import hashlib
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad

# 1. Seed the MT19937 PRNG
seed = os.urandom(16)
rng = random.Random()
rng.seed(seed)

# 2. Generate 624 outputs for the players
outputs = []
for _ in range(624):
    outputs.append(rng.getrandbits(32))

with open("eleven_numbers.bin", "wb") as f:
    for num in outputs:
        f.write(struct.pack("<I", num)) # Little-endian 32-bit integer

# 3. Generate the key from the *next* outputs
key_part1 = rng.getrandbits(32)
key_part2 = rng.getrandbits(32)
key_part3 = rng.getrandbits(32)
key_part4 = rng.getrandbits(32)
key = struct.pack("<IIII", key_part1, key_part2, key_part3, key_part4)

# 4. Encrypt the secret message
part2 = hashlib.sha256(b"eleven_seed:" + seed).hexdigest()[12:]
plaintext = f"This is a message from the upside down. Your flag part is: {part2}".encode()

cipher = AES.new(key, AES.MODE_CBC)
ciphertext = cipher.encrypt(pad(plaintext, AES.block_size))
iv = cipher.iv

with open("encrypted_message.bin", "wb") as f:
    f.write(iv)
    f.write(ciphertext)
```

```
print(f"✓ Stage 2 generated: elevens numbers.bin, encrypted message.bin")
print(f"🔒 Keep this secret! The seed was: {seed.hex()}")
print(f"🔒 Flag Part 2 is: {part2}")
PY
```

```
(ctf-env)-(radhika23s@kali)-[~/questCON_ctf]
$ python3 stage2_generator.py
```

```
✓ Stage 2 generated: elevens_numbers.bin, encrypted_message.bin
🔒 Keep this secret! The seed was: 764366c1c1695cc2e8605794a2e46cc0
🔒 Flag Part 2 is: 905d8af0d698
```

Stage 3: AES-CTR Keystream Reuse

- **Vulnerability:** AES in Counter (CTR) mode becomes a stream cipher. If the same key and nonce are ever reused, the exact same keystream is generated. This allows an attacker with a known plaintext ("crib") to recover the keystream and decrypt any other message.
- **Attack:** A known plaintext is hidden in the hint_partX.png images using LSB steganography. We extract this plaintext, XOR it with its corresponding ciphertext (msg001.ct) to recover the keystream, and then XOR that keystream with the target ciphertext (msg002.ct) to reveal the flag.
- **Solver:** solve_stage3.py

```
(ctf-env)-(radhika23s@kali)-[~/questCON_ctf]
$ cat > stage3_generator.py <<'PY'
#!/usr/bin/env python3
# stage3_generator.py: A Stranger Signal

import os
import hashlib
from PIL import Image
from Crypto.Cipher import AES
from Crypto.Util.Counter import new as new_counter

# 1. Generate a secret key and a (reused) nonce
key = os.urandom(16)
nonce = os.urandom(8) # 64-bit nonce for CTR

# 2. Prepare plaintexts
part3 = hashlib.sha256(b"stranger_things_key:" + key).hexdigest()[:12]
known_plaintext = b"From: Dr. Brenner <mbrenner@hkinslab.gov>"
target_plaintext = f"Subject: Project Eleven Status Update. The key part is: {part3}".encode()
other_plaintexts = [
    b"The gate is unstable. We need to close it.",
    b"Asset has breached containment. All personnel to lockdown.",
    b"Observation: Subject exhibits psychic resonance.",
    b"URGENT: Power fluctuations detected originating from the Upside Down.",
]

all_plaintexts = [known_plaintext, target_plaintext] + other_plaintexts

# 3. Encrypt all plaintexts with the same key and nonce
os.makedirs("intercepted_signals", exist_ok=True)
for i, pt in enumerate(all_plaintexts):
    counter = new_counter(64, prefix=nonce)
    cipher = AES.new(key, AES.MODE_CTR, counter=counter)
    ct = cipher.encrypt(pt)
    with open(f"intercepted_signals/signal_{i}.bin", "wb") as f:
        f.write(ct)

# 4. Hide the known plaintext in an image via LSB steganography
img = Image.new('RGB', (200, 50), color = 'black')
pixels = img.load()
binary_known_pt = ''.join(format(byte, '08b') for byte in known_plaintext)

data_index = 0
for y in range(img.height):
    for x in range(img.width):
```



```

        if data_index < len(binary_known_pt):
            r, g, b = pixels[x, y]
            # Embed in the LSB of the red channel

(ctf-env)-(radhika23s@kali)-[~/questCON_ctf]
$ python3 stage3_generator.py
✓ Stage 3 generated: intercepted_signals/, hint_image.png
😄 Keep this secret! The key was: c2eea6a6f28b855b03e5e2e19a6c11aa
😄 Flag Part 3 is: 9bd193964576

```

Conclusion

By successfully executing all three attacks, we recover the three parts of the flag. Assembling them gives us the final solution.

Final Flag: “questCON{5c8fd51d18da-905d8af0d698-9bd193964576}”

We will verify it

```

(ctf-env)-(radhika23s@kali)-[~/questCON_ctf]
$ # 1. Define the inner part of your NEW static flag.
FLAG_BODY="5c8fd51d18da-905d8af0d698-9bd193964576"

# 2. Calculate its SHA-256 hash and store it in a variable.
FINAL_HASH=$(echo -n "${FLAG_BODY}" | sha256sum | head -c 64)

# 3. Create the final 'verify_flag.py' script with the new hash.
cat > verify_flag.py <<PY
#!/usr/bin/env python3
import hashlib
import sys

# The correct hash is now permanently embedded here.
CORRECT_HASH = "${FINAL_HASH}"

# Check if the user provided a flag
if len(sys.argv) != 2:
    print(f"Usage: python3 {sys.argv[0]} questCON{{part1-part2-part3}}")
    sys.exit(1)

flag = sys.argv[1]

# Check the flag format
if not (flag.startswith("questCON(") and flag.endswith(")")):
    print("x Invalid flag format. It must be questCON{...}")
    sys.exit(1)

# Extract the part inside the braces
inner_flag = flag[9:-1]

# Hash the user's input
user_hash = hashlib.sha256(inner_flag.encode()).hexdigest()

# Compare the hashes
if user_hash == CORRECT_HASH:
    print("🎉 Congratulations! That's the correct flag!")
else:
    print("x Incorrect flag. The signal is wrong. Keep trying!")
PY

# 4. Make the script executable.
chmod +x verify_flag.py

# 5. Test the script with your correct flag.
echo "—— Verifying the final flag ——"
./verify_flag.py "questCON{${FLAG_BODY}}"
```

```
# 5. Test the script with your correct flag.  
echo "— Verifying the final flag —"  
./verify_flag.py "questCON${FLAG_BODY}"  
— Verifying the final flag —  
🎉 Congratulations! That's the correct flag!
```